SkillLink Project Documentation

1. Project overview

SkillLink is a Flutter marketplace application for discovering and booking trusted local service providers in Addis Ababa, Ethiopia.

The MVP supports two primary user journeys:

- Customers discover providers, view profiles, request bookings, and track their booking history.
- Service professionals submit provider registrations that can be reviewed by an administrator before being published.

The project is designed for local development, academic demonstration, and early MVP validation. It is not a payment platform and does not currently provide a production-grade account-management console.

2. Repository layout

```
skill_link/
├── lib/
│   ├── data/                    # Categories and demo provider data
│   ├── models/                  # Booking, provider, and registration models
│   ├── screens/                 # Application screens and workflows
│   ├── services/                # Firebase, storage, booking, and gateway
services
│   ├── theme/                   # Material 3 colors and theme
│   └── widgets/                 # Reusable UI components
├── test/                        # Flutter tests
├── external-admin-gateway/
│   ├── src/                     # Deno gateway implementation
│   ├── tests/                   # Gateway unit tests
│   └── supabase/functions/      # Supabase Edge Function entry point
├── functions/                   # Firebase-native callable alternative
├── firestore.rules              # Firestore authorization rules
├── storage.rules                # Firebase Storage authorization rules
├── pubspec.yaml                 # Flutter dependencies
└── docs/                        # Project documentation
```

3. Technology stack

Mobile application

- Flutter and Dart with sound null safety
- Material 3
- Firebase Authentication
- Cloud Firestore
- Firebase Storage

- `shared_preferences` for local booking and registration cache
- `image_picker` for profile images
- `http` for admin gateway requests

Administrative backend

- Supabase Edge Functions
- Deno and TypeScript
- `jose` for Firebase ID-token verification
- Firestore REST API
- Google service-account OAuth assertions created on the server

Firebase remains the source of truth for authentication and application data. Supabase is used only as an external server-side gateway for administrator registration review operations.

4. Application features

Customer features

- Email/password sign-up and sign-in
- Home screen with service categories and featured providers
- Provider search, category filtering, and sorting
- Provider profile and service details
- Booking request creation
- Booking history and status display
- Profile management

Provider features

- Provider registration form
- Service category, location, experience, pricing, availability, and description fields
- Optional profile image upload
- Registration status history
- Local persistence for demonstration resilience

Administrator features

- Administrator claim detection through the Firebase ID token
- Registration list retrieval
- Approval and rejection of pending registrations
- Rejection reason support
- Published-provider visibility after approval

5. Application startup and navigation

The application starts in `lib/main.dart` and performs these steps:

1. Initializes Flutter bindings.
2. Initializes Firebase using `firebase_options.dart`.
3. Initializes booking persistence.

4. Initializes provider-registration persistence.
5. Initializes authentication and token-claim tracking.
6. Displays the authentication gate.

Unauthenticated users see the authentication screen. Authenticated users see the main navigation destinations:

- Home
- Explore
- My Bookings
- Profile

The profile area exposes administrator functionality only when the verified Firebase ID token contains the custom claim `admin: true`.

6. Firebase data model

`users/{userId}`

User profile documents are owner-scoped. The rules require the document UID, authenticated UID, email, and customer role to remain consistent. Clients cannot change the role after creation.

Typical fields include:

```
uid
name
email
role: "customer"
profileImageUrl
createdAt
```

`providerRegistrations/{registrationId}`

Provider registrations are submitted as pending and unpublished.

Typical fields include:

```
id
reference
fullName
phone
email
location
category
professionalTitle
experienceYears
description
price
pricingUnit
```

```
availability
profileImageUrl
ownerUid
status: "pending" | "approved" | "rejected"
isPublished: true | false
createdAt
reviewedAt
reviewedByUid
rejectionReason
```

Public provider discovery queries only documents where:

```
status == "approved"
isPublished == true
```

Pending and rejected registrations are not public.

Bookings

Booking models and persistence are implemented in the Flutter booking service. Booking data should remain owner-scoped when extending the Firestore rules or adding new backend operations.

7. Security model

Client-side Firebase access

- Customers can read their own user profile.
- Customers can create and update their own profile within the allowed fields.
- Providers can create registrations owned by their authenticated UID.
- Registration owners can read their own registrations.
- Approved and published registrations can be read for public discovery.
- Clients cannot approve, reject, publish, or delete registrations.
- Firestore rules remain the enforcement boundary for direct mobile access.

The `role` field in a Firestore user profile is not an administrator security mechanism. Administrator authorization is based on the Firebase custom claim `admin: true`.

Admin gateway

The gateway:

1. Receives a Firebase bearer ID token.
2. Verifies its signature using Google's rotating Firebase public keys.
3. Verifies issuer, audience, algorithm, expiration, and subject.
4. Requires the verified `admin` claim to be exactly `true`.
5. Derives `reviewedByUid` from the verified token.
6. Reads and writes Firestore only from the server.

The gateway never trusts a UID, email address, or admin flag from the request body. Service-account credentials and Google OAuth access tokens must never be sent to Flutter or logged.

8. Admin gateway API

Base URL:

```
https://<SUPABASE_PROJECT_REF>.supabase.co/functions/v1/admin-registrations
```

All operations use `POST` and require:

```
Authorization: Bearer <firebase-id-token>
Content-Type: application/json
```

List registrations

```
POST /admin/registrations/list
```

Request body:

```
{}
```

The response contains dashboard-safe registration fields. Private fields that are not required by the dashboard should not be added to this response.

Review a registration

```
POST /admin/registrations/review
```

Approve:

```
{
  "registrationId": "registration-id",
  "decision": "approved"
}
```

Reject:

```
{
  "registrationId": "registration-id",
  "decision": "rejected",
  "rejectionReason": "Additional information is required."
}
```

Review operations are concurrency-safe. The gateway checks that the current registration is still pending and uses Firestore update-time preconditions. Conflicting reviews return HTTP `409`.

HTTP behavior

- `200`: successful list or review
- `204`: allowed CORS preflight
- `400`: malformed JSON or invalid request fields
- `401`: missing or invalid Firebase authentication
- `403`: non-admin token or disallowed origin
- `404`: unknown route or registration
- `405`: unsupported HTTP method
- `409`: registration was reviewed concurrently or is no longer pending
- `413`: request body exceeds the configured limit
- `500`: unexpected server-side failure

9. Local setup

Flutter application

Prerequisites:

- Flutter SDK compatible with the Dart constraint in `pubspec.yaml`
- Android Studio or another Flutter-compatible Android toolchain
- A configured Firebase project

From `skill_link/`:

```
flutter pub get
flutter analyze
flutter test
flutter run
```

Firebase platform configuration must be supplied through the normal Flutter configuration files. Do not commit private credentials or service-account files.

External admin gateway

Prerequisites:

- Deno

- Supabase CLI for remote deployment

From `external-admin-gateway/`:

```
deno check src\config.ts src\firestore.ts src\handler.ts src\auth.ts `
  supabase\functions\admin-registrations\index.ts tests\handler_test.ts
deno test tests\
deno fmt --check .
```

The tests use fake authentication and an in-memory registration store. They do not require production Firestore data or service-account credentials.

10. Supabase deployment configuration

The function is configured with `verify_jwt = false` because it validates Firebase tokens itself. This setting must not be interpreted as disabling authentication.

Required server-side secrets:

```
FIREBASE_PROJECT_ID
FIREBASE_SERVICE_ACCOUNT_JSON
```

Configure them through the Supabase Dashboard or Supabase CLI secret management. Never place the service-account JSON in source code, Flutter assets, Git, shell history, logs, or the request body.

The service account should receive only the minimum Firestore permissions needed for the gateway. The current deployment guidance uses the narrow Firestore datastore role rather than Owner, Editor, Firebase Admin, or Service Account Admin.

Deployment command:

```
supabase functions deploy admin-registrations `
  --project-ref <SUPABASE_PROJECT_REF> `
  --no-verify-jwt
```

After deployment, verify the function status without printing secret values:

```
supabase functions list --project-ref <SUPABASE_PROJECT_REF>
supabase secrets list --project-ref <SUPABASE_PROJECT_REF>
```

Only safe authentication probes should be used initially:

- no `Authorization` header: expect `401`
- invalid Firebase token: expect `401`

Do not perform approval or rejection requests until the deployment, secrets, IAM permissions, and authenticated administrator flow have been independently reviewed.

11. Configuration and operational limitations

- The gateway requires `FIREBASE_PROJECT_ID`; it fails closed when the value is missing or invalid.
- The gateway requires `FIREBASE_SERVICE_ACCOUNT_JSON` at runtime.
- The Flutter client currently points to the configured admin gateway URL in `AdminService`; change this only through an intentional deployment configuration update.
- Automated gateway tests do not prove live Google key retrieval, live OAuth exchange, or live Firestore REST behavior.
- Docker/Podman is required for some local Supabase commands, but is not required for remote function deployment.
- The Firebase-native callable Functions in `functions/index.js` remain an alternative implementation and are not a replacement for the external gateway unless Firebase Functions deployment is enabled.
- Real payments, provider payouts, messaging, ratings, and production observability are outside the current MVP scope.

12. Recommended next steps

1. Keep Firebase security rules deployed and reviewed independently of the gateway.
2. Verify the two Supabase secret names without exposing their values.
3. Test missing and invalid authentication against the deployed endpoint.
4. Test authenticated listing with a controlled administrator account.
5. Add a non-production environment before testing review writes.
6. Connect review status refresh behavior to the Flutter dashboard only after the gateway contract is stable.
7. Rotate service-account credentials immediately if they are ever exposed.